

only 6000 die rolls.

Displaying animation frames to the screen is a relatively slow *input–output-bound* operation compared to the die rolls, which occur at the computer’s super fast CPU speeds. If we roll only one die per animation frame, we won’t be able to run a large number of rolls in a reasonable amount of time. Also, for small numbers of rolls, you’re unlikely to see the die percentages converge on their expected 16.667% of the total rolls.

To see the law of large numbers in action, you can increase the execution speed by rolling the die more times per animation frame. Consider the following command:

```
ipython RollDieDynamic.py 10000 600
```

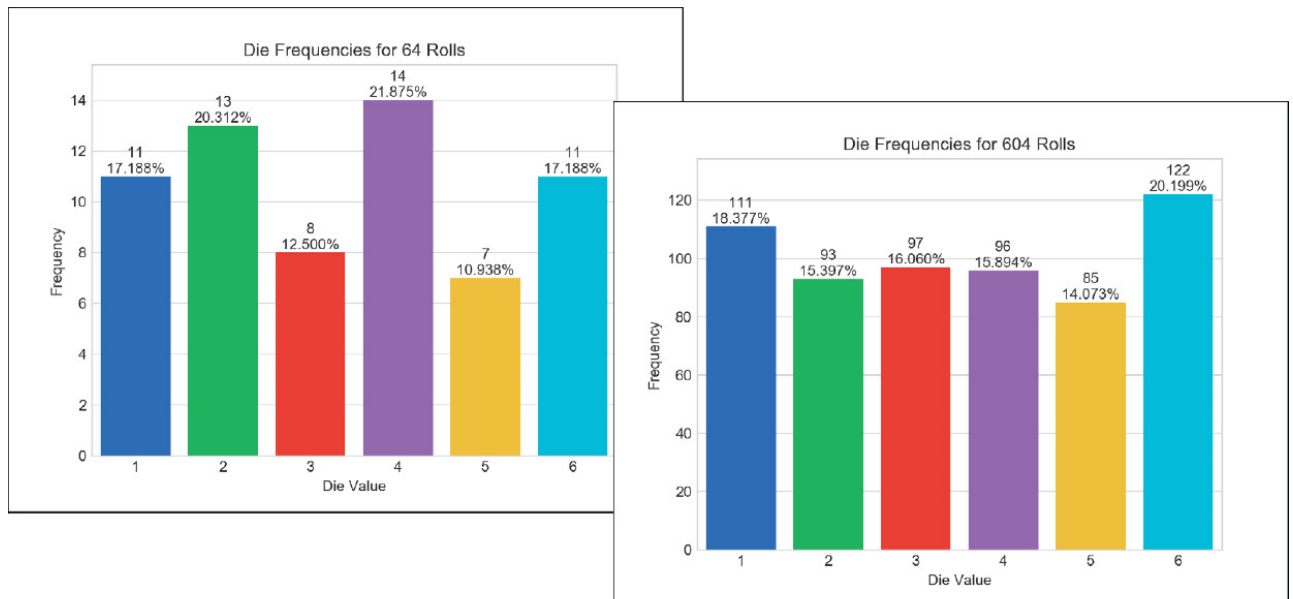
In this case, `FuncAnimation` will call our `update` function 10,000 times, performing 600 rolls-per-frame for a total of 6,000,000 rolls. On our system, this took about 5.55 minutes (10,000 frames / 30 frames-per-second / 60 seconds-per-minute), but displayed approximately 18,000 rolls-per-second (30 frames-per-second * 600 rolls-per-frame), so we could quickly see the frequencies and percentages converge on their expected values of about 1,000,000 rolls per face and 16.667% per face.

Experiment with the numbers of rolls and frames until you feel that the program is helping you visualize the results most effectively. It’s fun and informative to watch it run and to tweak it until you’re satisfied with the animation quality.

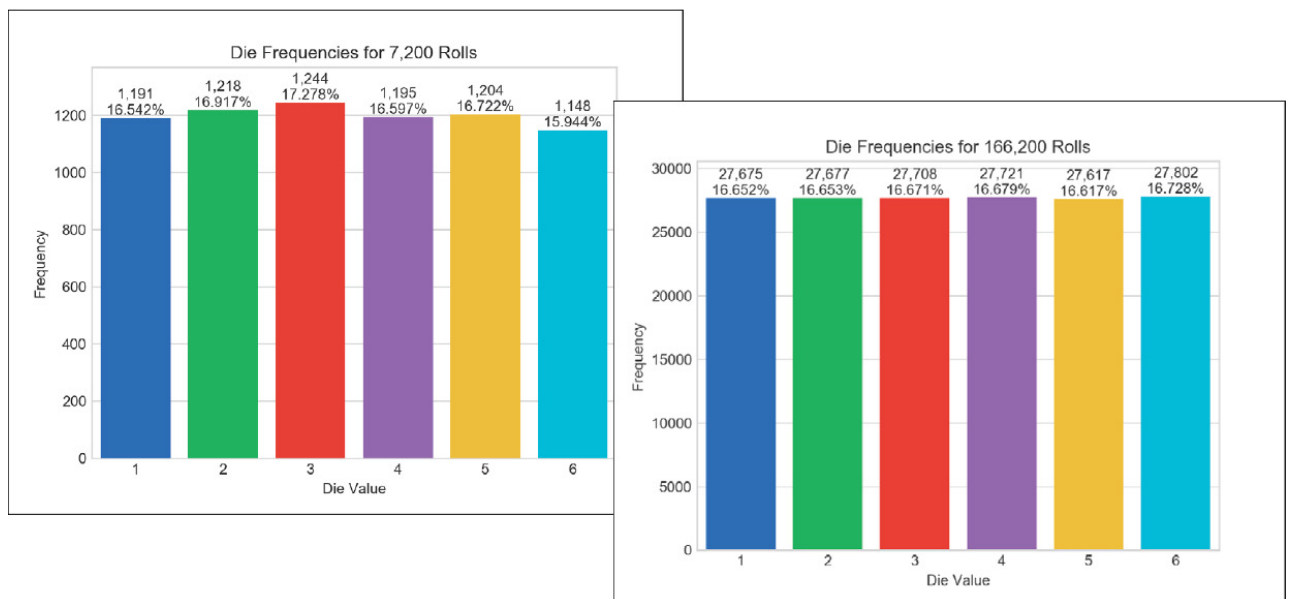
Sample Executions

We took the following four screen captures during each of two sample executions. In the first, the screens show the graph after just 64 die rolls, then again after 604 of the 6000 total die rolls. Run this script live to see over time how the bars update dynamically. In the second execution, the screen captures show the graph after 7200 die rolls and again after 166,200 out of the 6,000,000 rolls. With more rolls, you can see the percentages closing in on their expected values of 16.667% as predicted by the law of large numbers.

Execute 6000 animation frames rolling the die once per frame:
`ipython RollDieDynamic.py 6000 1`



Execute 10,000 animation frames rolling the die 600 times per frame:
`ipython RollDieDynamic.py 10000 600`



.4.2 Implementing a Dynamic Visualization

The script we present in this section uses the same Seaborn and Matplotlib features shown in the previous chapter's Intro to Data Science section. We reorganized the code for use with Matplotlib's *animation* capabilities.

Importing the Matplotlib `animation` Module

We focus primarily on the new features used in this example. Line 3 imports the Matplotlib `animation` module.

[lick here to view code image](#)

```
1 # RollDieDynamic.py
```

```

2 """Dynamically graphing frequencies of die rolls."""
3 from matplotlib import animation
4 import matplotlib.pyplot as plt
5 import random
6 import seaborn as sns
7 import sys
8

```

Function update

Lines 9–27 define the `update` function that `FuncAnimation` calls once per animation frame. This function must provide at least one argument. Lines 9–10 show the beginning of the function definition. The parameters are:

- `frame_number`—The next value from `FuncAnimation`’s `frames` argument, which we’ll discuss momentarily. Though `FuncAnimation` requires the `update` function to have this parameter, we do not use it in this `update` function.
- `rolls`—The number of die rolls per animation frame.
- `faces`—The die face values used as labels along the graph’s *x*-axis.
- `frequencies`—The list in which we summarize the die frequencies.

We discuss the rest of the function’s body in the next several subsections.

[lick here to view code image](#)

```

9 def update(frame_number, rolls, faces, frequencies):
10     """Configures bar plot contents for each animation frame."""

```

Function update: Rolling the Die and Updating the frequencies List

Lines 12–13 roll the die `rolls` times and increment the appropriate `frequencies` element for each roll. Note that we subtract 1 from the die value (1 through 6) before incrementing the corresponding `frequencies` element—as you’ll see, `frequencies` is a six-element list (defined in line 36), so its indices are 0 through 5.

[lick here to view code image](#)

```

11     # roll die and update frequencies
12     for i in range(rolls):

```

```
13 frequencies[random.randrange(1, 7) - 1] += 1
14
```

Function update: Configuring the Bar Plot and Text

Line 16 in function `update` calls the `matplotlib.pyplot` module's `cla` (clear axes) function to remove the existing bar plot elements before drawing new ones for the current animation frame. We discussed the code in lines 17–27 in the previous chapter's Intro to Data Science section. Lines 17–20 create the bars, set the bar plot's title, set the *x*- and *y*-axis labels and scale the plot to make room for the frequency and percentage text above each bar. Lines 23–27 display the frequency and percentage text.

[lick here to view code image](#)

```
15 # reconfigure plot for updated die frequencies
16 plt.cla() # clear old contents contents of current Figure
17 axes = sns.barplot(faces, frequencies, palette='bright') # ne
18 axes.set_title(f'Die Frequencies for {sum(frequencies):,} Rolls')
19 axes.set_xlabel='Die Value', ylabel='Frequency'
20 axes.set_ylim(top=max(frequencies) * 1.10) # scale y-axis by
21
22 # display frequency & percentage above each patch (bar)
23 for bar, frequency in zip(axes.patches, frequencies):
24     text_x = bar.get_x() + bar.get_width() / 2.0
25     text_y = bar.get_height()
26     text = f'{frequency:,\n}{frequency / sum(frequencies):.3%}'
27     axes.text(text_x, text_y, text, ha='center', va='bottom')
28
```

Variables Used to Configure the Graph and Maintain State

Lines 30 and 31 use the `sys` module's `argv` list to get the script's command-line arguments. Line 33 specifies the Seaborn 'whitegrid' style. Line 34 calls the `matplotlib.pyplot` module's `figure` function to get the `Figure` object in which `FuncAnimation` displays the animation. The function's argument is the window's title. As you'll soon see, this is one of `FuncAnimation`'s required arguments. Line 35 creates a list containing the die face values 1–6 to display on the plot's *x*-axis. Line 36 creates the six-element `frequencies` list with each element initialized to 0—we update this list's counts with each die roll.

[lick here to view code image](#)

```
29 # read command-line arguments for number of frames and rolls per fra
```

```

30 number_of_frames    = int(sys.argv[1])
31 rolls_per_frame     = int(sys.argv[2])
32
33 sns.set_style('whitegrid') # white background with gray grid lines
34 figure = plt.figure('Rolling a Six-Sided Die') # Figure for animation
35 values = list(range(1, 7)) # die faces for display on x-axis
36 frequencies = [0] * 6 # six-element list of die frequencies
37

```

alling the animation Module's FuncAnimation Function

Lines 39–41 call the Matplotlib animation module's `FuncAnimation` function to update the bar chart dynamically. The function returns an object representing the animation. Though this is not used explicitly, you *must* store the reference to the animation; otherwise, Python immediately terminates the animation and returns its memory to the system.

[lick here to view code image](#)

```

38 # configure and start animation that calls function update
39 die_animation = animation.FuncAnimation(
40     figure, update, repeat=False, frames=number_of_frames, interval=
41     fargs=(rolls_per_frame, values, frequencies))
42
43 plt.show() # display window

```

`FuncAnimation` has two required arguments:

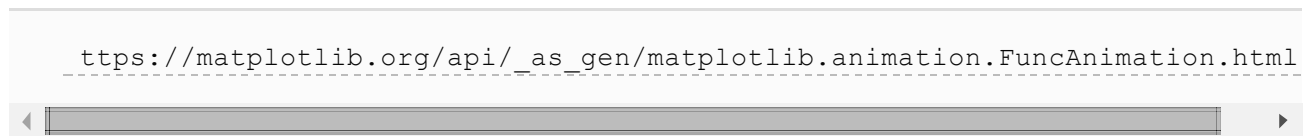
- **figure**—the `Figure` object in which to display the animation, and
- **update**—the function to call once per animation frame.

In this case, we also pass the following optional keyword arguments:

- **repeat**—`False` terminates the animation after the specified number of frames. If `True` (the default), when the animation completes it restarts from the beginning.
- **frames**—The total number of animation frames, which controls how many times `FuncAnimation` calls `update`. Passing an integer is equivalent to passing a range—for example, `600` means `range(600)`. `FuncAnimation` passes one value from this range as the first argument in each call to `update`.

- **interval**—The number of milliseconds (33, in this case) between animation frames (the default is 200). After each call to `update`, `FuncAnimation` waits 33 milliseconds before making the next call.
- **fargs** (short for “function arguments”)—A tuple of other arguments to pass to the function you specified in `FuncAnimation`’s second argument. The arguments you specify in the `fargs` tuple correspond to `update`’s parameters `rolls`, `faces` and `frequencies` (line 9).

For a list of `FuncAnimation`’s other optional arguments, see



Finally, line 43 displays the window.

6.5 WRAP-UP

In this chapter, we discussed Python’s dictionary and set collections. We said what a dictionary is and presented several examples. We showed the syntax of key–value pairs and showed how to use them to create dictionaries with comma-separated lists of key–value pairs in curly braces, `{ }`. You also created dictionaries with dictionary comprehensions.

You used square brackets, `[ ]`, to retrieve the value corresponding to a key, and to insert and update key–value pairs. You also used the dictionary method `update` to change a key’s associated value. You iterated through a dictionary’s keys, values and items.

You created sets of unique immutable values. You compared sets with the comparison operators, combined sets with set operators and methods, changed sets’ values with the mutable set operations and created sets with set comprehensions. You saw that sets are mutable. `Frozensets` are immutable, so they can be used as set and `frozenset` elements.

In the Intro to Data Science section, we continued our visualization introduction by presenting the die-rolling simulation with a *dynamic* bar plot to make the law of large numbers “come alive.” In addition, to the Seaborn and Matplotlib features shown in the previous chapter’s Intro to Data Science section, we used Matplotlib’s `FuncAnimation` function to control a frame-by-frame animation. `FuncAnimation` called a function we defined that specified what to display in each animation frame.

n the next chapter, we discuss array-oriented programming with the popular NumPy library. As you'll see, NumPy's `ndarray` collection can be up to two orders of magnitude faster than performing many of the same operations with Python's built-in lists. This power will come in handy for today's big data applications.

. Array-Oriented Programming with NumPy

Objectives

In this chapter you'll:

- Learn how arrays differ from lists.
- Use the `numpy` module's high-performance `ndarrays`.
- Compare list and `ndarray` performance with the IPython `%timeit` magic.
- Use `ndarrays` to store and retrieve data efficiently.
- Create and initialize `ndarrays`.
- Refer to individual `ndarray` elements.
- Iterate through `ndarrays`.
- Create and manipulate multidimensional `ndarrays`.
- Perform common `ndarray` manipulations.
- Create and manipulate pandas one-dimensional `Series` and two-dimensional `DataFrames`.
- Customize `Series` and `DataFrame` indices.
- Calculate basic descriptive statistics for data in a `Series` and a `DataFrame`.
- Customize floating-point number precision in pandas output formatting.

Outline

.1	Introduction
.2	Creating arrays from Existing Data
.3	array Attributes
.4	Filling arrays with Specific Values
.5	Creating arrays from Ranges
.6	List vs. array Performance: Introducing <code>%timeit</code>
.7	array Operators
.8	NumPy Calculation Methods
.9	Universal Functions
.10	Indexing and Slicing
.11	Views: Shallow Copies
.12	Deep Copies
.13	Reshaping and Transposing
.14	Intro to Data Science: pandas Series and DataFrames
.14.1	pandas Series
.14.2	DataFrames
.15	Wrap-Up

7.1 INTRODUCTION

The **NumPy (Numerical Python)** library first appeared in 2006 and is the preferred Python array implementation. It offers a high-performance, richly functional *n*-dimensional array type called **`ndarray`**, which from this point forward we'll refer to by its synonym, `array`. NumPy is one of the many open-source libraries that the Anaconda Python distribution installs. Operations on `arrays` are up to two orders of

magnitude faster than those on lists. In a big-data world in which applications may do massive amounts of processing on vast amounts of array-based data, this performance advantage can be critical. According to `libraries.io`, over 450 Python libraries depend on NumPy. Many popular data science libraries such as Pandas, SciPy (Scientific Python) and Keras (for deep learning) are built on or depend on NumPy.

In this chapter, we explore `array`'s basic capabilities. Lists can have multiple dimensions. You generally process multi-dimensional lists with nested loops or list comprehensions with multiple `for` clauses. A strength of NumPy is “array-oriented programming,” which uses functional-style programming with *internal* iteration to make array manipulations concise and straightforward, eliminating the kinds of bugs that can occur with the *external* iteration of explicitly programmed loops.

In this chapter's Intro to Data Science section, we begin our multi-section introduction to the *pandas* library that you'll use in many of the data science case study chapters. Big data applications often need more flexible collections than NumPy's `arrays`—collections that support mixed data types, custom indexing, missing data, data that's not structured consistently and data that needs to be manipulated into forms appropriate for the databases and data analysis packages you use. We'll introduce *pandas* array-like one-dimensional `Series` and two-dimensional `DataFrames` and begin demonstrating their powerful capabilities. After reading this chapter, you'll be familiar with four array-like collections—lists, `arrays`, `Series` and `DataFrames`. We'll add a fifth—tensors—in the “Deep Learning” chapter.

7.2 CREATING ARRAYS FROM EXISTING DATA

The NumPy documentation recommends importing the **numpy module** as `np` so that you can access its members with `"np."`:

```
In [1]: import numpy as np
```

The `numpy` module provides various functions for creating `arrays`. Here we use the **array** function, which receives as an argument an `array` or other collection of elements and returns a new `array` containing the argument's elements. Let's pass a list:

[lick here to view code image](#)

```
In [2]: numbers = np.array([2, 3, 5, 7, 11])
```

The `array` function copies its argument's contents into the `array`. Let's look at the type of object that function `array` returns and display its contents:

[lick here to view code image](#)

```
In [3]: type(numbers)
Out[3]: numpy.ndarray

In [4]: numbers
Out[4]: array([ 2,  3,  5,  7, 11])
```

Note that the *type* is `numpy.ndarray`, but all arrays are output as “array.” When outputting an array, NumPy separates each value from the next with a comma and a space and *right-aligns* all the values using the same field width. It determines the field width based on the value that occupies the *largest* number of character positions. In this case, the value 11 occupies the two character positions, so all the values are formatted in two-character fields. That's why there's a leading space between the [and 2.

Multidimensional Arguments

The `array` function copies its argument's dimensions. Let's create an array from a two-row-by-three-column list:

[lick here to view code image](#)

```
In [5]: np.array([[1, 2, 3], [4, 5, 6]])
Out[5]:
array([[1, 2, 3],
       [4, 5, 6]])
```

NumPy auto-formats arrays, based on their number of dimensions, aligning the columns within each row.

7.3 ARRAY ATTRIBUTES

An array object provides **attributes** that enable you to discover information about its structure and contents. In this section we'll use the following arrays:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: integers = np.array([[1, 2, 3], [4, 5, 6]])

In [3]: integers
Out[3]:
array([[1, 2, 3],
       [4, 5, 6]])

In [4]: floats = np.array([0.0, 0.1, 0.2, 0.3, 0.4])

In [5]: floats
Out[5]: array([ 0. , 0.1, 0.2, 0.3, 0.4])
```

NumPy does not display trailing os to the right of the decimal point in floating-point values.

Determining an array's Element Type

The `array` function determines an array's element type from its argument's elements. You can check the element type with an array's `dtype` attribute:

[lick here to view code image](#)

```
In [6]: integers.dtype
Out[6]: dtype('int64') # int32 on some platforms

In [7]: floats.dtype
Out[7]: dtype('float64')
```

As you'll see in the next section, various array-creation functions receive a `dtype` keyword argument so you can specify an array's element type.

For performance reasons, NumPy is written in the C programming language and uses C's data types. By default, NumPy stores integers as the NumPy type `int64` values—which correspond to 64-bit (8-byte) integers in C—and stores floating-point numbers as the NumPy type `float64` values—which correspond to 64-bit (8-byte) floating-point values in C. In our examples, most commonly you'll see the types `int64`, `float64`, `bool` (for Boolean) and `object` for non-numeric data (such as strings). The complete list of supported types is at

<https://docs.scipy.org/doc/numpy/user/basics.types.html>.

Determining an array's Dimensions

The attribute **ndim** contains an array's number of dimensions and the attribute **shape** contains a *tuple* specifying an array's dimensions:

[lick here to view code image](#)

```
In [8]: integers.ndim
Out[8]: 2

In [9]: floats.ndim
Out[9]: 1

In [10]: integers.shape
Out[10]: (2, 3)

In [11]: floats.shape
Out[11]: (5,)
```

Here, `integers` has 2 rows and 3 columns (6 elements) and `floats` is one-dimensional, so snippet [11] shows a one-element tuple (indicated by the comma) containing `floats`' number of elements (5).

Determining an array's Number of Elements and Element Size

You can view an array's total number of elements with the attribute **size** and the number of bytes required to store each element with **itemsize**:

[lick here to view code image](#)

```
In [12]: integers.size
Out[12]: 6

In [13]: integers.itemsize # 4 if C compiler uses 32-bit ints
Out[13]: 8

In [14]: floats.size
Out[14]: 5

In [15]: floats.itemsize
Out[15]: 8
```

Note that `integers`' size is the product of the `shape` tuple's values—two rows of three elements each for a total of six elements. In each case, `itemsize` is 8 because `integers` contains `int64` values and `floats` contains `float64` values, which each occupy 8 bytes.

Iterating Through a Multidimensional array's Elements

You'll generally manipulate arrays using concise functional-style programming techniques. However, because arrays are *iterable*, you can use external iteration if you'd like:

[lick here to view code image](#)

```
In [16]: for row in integers:
...:     for column in row:
...:         print(column, end=' ')
...:     print()
...:
1  2  3
4  5  6
```

You can iterate through a multidimensional array as if it were one-dimensional by using its **flat** attribute:

[lick here to view code image](#)

```
In [17]: for i in integers.flat:
...:     print(i, end=' ')
...:
1  2  3  4  5  6
```

7.4 FILLING ARRAYS WITH SPECIFIC VALUES

NumPy provides functions **zeros**, **ones** and **full** for creating arrays containing 0s, 1s or a specified value, respectively. By default, **zeros** and **ones** create arrays containing `float64` values. We'll show how to customize the element type momentarily. The first argument to these functions must be an integer or a tuple of integers specifying the desired dimensions. For an integer, each function returns a one-dimensional array with the specified number of elements:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: np.zeros(5)
Out[2]: array([ 0.,  0.,  0.,  0.,  0.])
```

For a tuple of integers, these functions return a multidimensional array with the specified dimensions. You can specify the array's element type with the `zeros` and `ones` function's `dtype` keyword argument:

[lick here to view code image](#)

```
In [3]: np.ones((2, 4), dtype=int)
Out[3]:
array([[1, 1, 1, 1],
       [1, 1, 1, 1]])
```

The array returned by `full` contains elements with the second argument's value and type:

[lick here to view code image](#)

```
In [4]: np.full((3, 5), 13)
Out[4]:
array([[13, 13, 13, 13, 13],
       [13, 13, 13, 13, 13],
       [13, 13, 13, 13, 13]])
```

7.5 CREATING ARRAYS FROM RANGES

NumPy provides optimized functions for creating arrays from ranges. We focus on simple evenly spaced integer and floating-point ranges, but NumPy also supports nonlinear ranges.¹

¹ <https://docs.scipy.org/doc/numpy/reference/routines.array-recreation.html>.

Creating Integer Ranges with `arange`

Let's use NumPy's `arange` function to create integer ranges—similar to using built-in function `range`. In each case, `arange` first determines the resulting array's number of elements, allocates the memory, then stores the specified range of values in the array:

[lick here to view code image](#)

```
In [1]: import numpy as np
```

```
In [2]: np.arange(5)
Out[2]: array([0, 1, 2, 3, 4])

In [3]: np.arange(5, 10)
Out[3]: array([5, 6, 7, 8, 9])

In [4]: np.arange(10, 1, -2)
Out[4]: array([10, 8, 6, 4, 2])
```

Though you can create arrays by passing ranges as arguments, always use `arange` as it's optimized for arrays. Soon we'll show how to determine the execution time of various operations so you can compare their performance.

Creating Floating-Point Ranges with `linspace`

You can produce evenly spaced floating-point ranges with NumPy's `linspace` function. The function's first two arguments specify the starting and ending values in the range, and the ending value *is included* in the array. The optional keyword argument `num` specifies the number of evenly spaced values to produce—this argument's default value is 50:

[lick here to view code image](#)

```
In [5]: np.linspace(0.0, 1.0, num=5)
Out[5]: array([ 0. ,  0.25,  0.5 ,  0.75,  1.  ])
```

Reshaping an array

You also can create an array from a range of elements, then use array method `reshape` to transform the one-dimensional array into a multidimensional array. Let's create an array containing the values from 1 through 20, then reshape it into four rows by five columns:

[lick here to view code image](#)

```
In [6]: np.arange(1, 21).reshape(4, 5)
Out[6]:
array([[ 1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10],
       [11, 12, 13, 14, 15],
       [16, 17, 18, 19, 20]])
```

Note the *chained method calls* in the preceding snippet. First, `arange` produces an

array containing the values 1–20. Then we call `reshape` on that array to get the 4-by-5 array that was displayed.

You can reshape any array, provided that the new shape has the *same* number of elements as the original. So a six-element one-dimensional array can become a 3-by-2 or 2-by-3 array, and vice versa, but attempting to reshape a 15-element array into a 4-by-4 array (16 elements) causes a `ValueError`.

Displaying Large arrays

When displaying an array, if there are 1000 items or more, NumPy drops the middle rows, columns or both from the output. The following snippets generate 100,000 elements. The first case shows all four rows but only the first and last three of the 25,000 columns. The notation `...` represents the missing data. The second case shows the first and last three of the 100 rows, and the first and last three of the 1000 columns:

[lick here to view code image](#)

```
In [7]: np.arange(1, 100001).reshape(4, 25000)
Out[7]:
array([[      1,       2,       3, ..., 24998, 24999, 25000],
       [ 25001, 25002, 25003, ..., 49998, 49999, 50000],
       [ 50001, 50002, 50003, ..., 74998, 74999, 75000],
       [ 75001, 75002, 75003, ..., 99998, 99999, 100000]])

In [8]: np.arange(1, 100001).reshape(100, 1000)
Out[8]:
array([[      1,       2,       3, ...,   998,   999,  1000],
       [ 1001,  1002,  1003, ...,  1998,  1999,  2000],
       [ 2001,  2002,  2003, ...,  2998,  2999,  3000],
       ...,
       [ 97001,  97002,  97003, ...,  97998,  97999,  98000],
       [ 98001,  98002,  98003, ...,  98998,  98999,  99000],
       [ 99001,  99002,  99003, ...,  99998,  99999, 100000]])
```

7.6 LIST VS. ARRAY PERFORMANCE: INTRODUCING %TIMEIT

Most array operations execute *significantly* faster than corresponding list operations. To demonstrate, we'll use the IPython `%timeit` magic command, which times the *average* duration of operations. Note that the times displayed on your system may vary from what we show here.

Timing the Creation of a List Containing Results of 6,000,000 Die Rolls

We've demonstrated rolling a six-sided die 6,000,000 times. Here, let's use the `random` module's `randrange` function with a list comprehension to create a list of six million die rolls and time the operation using `%timeit`. Note that we used the line-continuation character (`\`) to split the statement in snippet [2] over two lines:

[lick here to view code image](#)

```
In [1]: import random

In [2]: %timeit rolls_list = \
...:     [random.randrange(1, 7) for i in range(0, 6_000_000)]
6.29 s ± 119 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

By default, `%timeit` executes a statement in a loop, and it runs the loop *seven* times. If you do not indicate the number of loops, `%timeit` chooses an appropriate value. In our testing, operations that on average took more than 500 milliseconds iterated only once, and operations that took fewer than 500 milliseconds iterated 10 times or more.

After executing the statement, `%timeit` displays the statement's *average* execution time, as well as the standard deviation of all the executions. On average, `%timeit` indicates that it took 6.29 seconds (s) to create the list with a standard deviation of 119 milliseconds (ms). In total, the preceding snippet took about 44 seconds to run the snippet seven times.

Timing the Creation of an `array` Containing Results of 6,000,000 Die Rolls

Now, let's use the **`randint` function** from the **`numpy.random` module** to create an array of 6,000,000 die rolls

[lick here to view code image](#)

```
In [3]: import numpy as np

In [4]: %timeit rolls_array = np.random.randint(1, 7, 6_000_000)
72.4 ms ± 635 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

On average, `%timeit` indicates that it took only 72.4 *milliseconds* with a standard deviation of 635 *microseconds* (µs) to create the array. In total, the preceding snippet took just under half a second to execute on our computer—about 1/100th of the time

snippet [2] took to execute. The operation is *two orders of magnitude faster* with array!

60,000,000 and 600,000,000 Die Rolls

Now, let's create an array of 60,000,000 die rolls:

[lick here to view code image](#)

```
In [5]: %timeit rolls_array = np.random.randint(1, 7, 60_000_000)
873 ms ± 29.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

On average, it took only 873 milliseconds to create the array.

Finally, let's do 600,000,000 million die rolls:

[lick here to view code image](#)

```
In [6]: %timeit rolls_array = np.random.randint(1, 7, 600_000_000)
10.1 s ± 232 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

It took about 10 seconds to create 600,000,000 elements with NumPy vs. about 6 seconds to create only 6,000,000 elements with a list comprehension.

Based on these timing studies, you can see clearly why arrays are preferred over lists for compute-intensive operations. In the data science case studies, we'll enter the performance-intensive worlds of big data and AI. We'll see how clever hardware, software, communications and algorithm designs combine to meet the often enormous computing challenges of today's applications.

Customizing the %timeit Iterations

The number of iterations within each %timeit loop and the number of loops are customizable with the `-n` and `-r` options. The following executes snippet [4]'s statement three times per loop and runs the loop twice: ²

² For most readers, using %timeits default settings should be fine.

[lick here to view code image](#)

```
In [7]: %timeit -n3 -r2 rolls_array = np.random.randint(1, 7, 6_000_000)
85.5 ms ± 5.32 ms per loop (mean ± std. dev. of 2 runs, 3 loops each)
```

Other IPython Magics

IPython provides dozens of magics for a variety of tasks—for a complete list, see the IPython magics documentation.³ Here are a few helpful ones:

3

<http://ipython.readthedocs.io/en/stable/interactive/magics.html>.

- `%load` to read code into IPython from a local file or URL.
- `%save` to save snippets to a file.
- `%run` to execute a `.py` file from IPython.
- `%precision` to change the default floating-point precision for IPython outputs.
- `%cd` to change directories without having to exit IPython first.
- `%edit` to launch an external editor—handy if you need to modify more complex snippets.
- `%history` to view a list of all snippets and commands you’ve executed in the current IPython session.

7.7 ARRAY OPERATORS

NumPy provides many operators which enable you to write simple expressions that perform operations on entire `arrays`. Here, we demonstrate arithmetic between `arrays` and numeric values and between `arrays` of the same shape.

Arithmetic Operations with `arrays` and Individual Numeric Values

First, let’s perform *element-wise arithmetic* with `arrays` and numeric values by using arithmetic operators and augmented assignments. Element-wise operations are applied to every element, so snippet [4] multiplies every element by 2 and snippet [5] cubes every element. Each returns a *new* `array` containing the result:

[lick here to view code image](#)

```
In [1]: import numpy as np
```

```
In [2]: numbers = np.arange(1, 6)

In [3]: numbers
Out[3]: array([1, 2, 3, 4, 5])

In [4]: numbers * 2
Out[4]: array([ 2,  4,  6,  8, 10])

In [5]: numbers ** 3
Out[5]: array([ 1,  8, 27, 64, 125])

In [6]: numbers # numbers is unchanged by the arithmetic operators
Out[6]: array([1, 2, 3, 4, 5])
```

Snippet [6] shows that the arithmetic operators did not modify `numbers`. Operators `+` and `*` are *commutative*, so snippet [4] could also be written as `2 * numbers`.

Augmented assignments *modify* every element in the left operand.

[lick here to view code image](#)

```
In [7]: numbers += 10

In [8]: numbers
Out[8]: array([11, 12, 13, 14, 15])
```

Broadcasting

Normally, the arithmetic operations require as operands two arrays of the *same size and shape*. When one operand is a single value, called a **scalar**, NumPy performs the element-wise calculations as if the scalar were an array of the same shape as the other operand, but with the scalar value in all its elements. This is called **broadcasting**. Snippets [4], [5] and [7] each use this capability. For example, snippet [4] is equivalent to:

[lick here to view code image](#)

```
numbers * [2, 2, 2, 2, 2]
```

Broadcasting also can be applied between arrays of different sizes and shapes, enabling some concise and powerful manipulations. We'll show more examples of broadcasting later in the chapter when we introduce NumPy's universal functions.

Arithmetic Operations Between arrays

You may perform arithmetic operations and augmented assignments between arrays of the *same* shape. Let's multiply the one-dimensional arrays `numbers` and `numbers2` (created below) that each contain five elements:

[lick here to view code image](#)

```
In [9]: numbers2 = np.linspace(1.1, 5.5, 5)

In [10]: numbers2
Out[10]: array([ 1.1,  2.2,  3.3,  4.4,  5.5])

In [11]: numbers * numbers2
Out[11]: array([ 12.1,  26.4,  42.9,  61.6,  82.5])
```

The result is a new array formed by multiplying the arrays *element-wise* in each operand—`11 * 1.1`, `12 * 2.2`, `13 * 3.3`, etc. Arithmetic between arrays of integers and floating-point numbers results in an array of floating-point numbers.

Comparing arrays

You can compare arrays with individual values and with other arrays. Comparisons are performed *element-wise*. Such comparisons produce arrays of Boolean values in which each element's `True` or `False` value indicates the comparison result:

[lick here to view code image](#)

```
In [12]: numbers
Out[12]: array([11, 12, 13, 14, 15])

In [13]: numbers >= 13
Out[13]: array([False, False,  True,  True,  True])

In [14]: numbers2
Out[14]: array([ 1.1,  2.2,  3.3,  4.4,  5.5])

In [15]: numbers2 < numbers
Out[15]: array([ True,  True,  True,  True,  True])

In [16]: numbers == numbers2
Out[16]: array([False, False, False, False, False])

In [17]: numbers == numbers
Out[17]: array([ True,  True,  True,  True,  True])
```

Snippet [13] uses broadcasting to determine whether each element of `numbers` is greater than or equal to 13. The remaining snippets compare the corresponding elements of each `array` operand.

7.8 NUMPY CALCULATION METHODS

An `array` has various methods that perform calculations using its contents. By default, these methods ignore the `array`'s shape and use *all* the elements in the calculations. For example, calculating the mean of an `array` totals all of its elements regardless of its shape, then divides by the total number of elements. You can perform these calculations on each dimension as well. For example, in a two-dimensional array, you can calculate each row's mean and each column's mean.

Consider an `array` representing four students' grades on three exams:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: grades = np.array([[87,    96,  70], [100,  87,  90],
...:                      [ 94,    77,  90], [100,  81,  82]])
...:

In [3]: grades
Out[3]:
array([[ 87,   96,   70],
       [100,   87,   90],
       [ 94,   77,   90],
       [100,   81,   82]])
```

We can use methods to calculate **sum**, **min**, **max**, **mean**, **std** (standard deviation) and **var** (variance)—each is a functional-style programming *reduction*:

[lick here to view code image](#)

```
In [4]: grades.sum()
Out[4]: 1054

In [5]: grades.min()
Out[5]: 70

In [6]: grades.max()
Out[6]: 100

In [7]: grades.mean()
```

```
Out[7]: 87.83333333333333
```

```
In [8]: grades.std()
```

```
Out[8]: 8.792357792739987
```

```
In [9]: grades.var()
```

```
Out[9]: 77.30555555555556
```

Calculations by Row or Column

Many calculation methods can be performed on specific `array` dimensions, known as the `array`'s *axes*. These methods receive an `axis` keyword argument that specifies which dimension to use in the calculation, giving you a quick way to perform calculations by row or column in a two-dimensional `array`.

Assume that you want to calculate the average grade on each *exam*, represented by the columns of `grades`. Specifying `axis=0` performs the calculation on all the *row* values within each column:

[lick here to view code image](#)

```
In [10]: grades.mean(axis=0)
```

```
Out[10]: array([95.25, 85.25, 83.  ])
```

So `95.25` above is the average of the first column's grades (87, 100, 94 and 100), `85.25` is the average of the second column's grades (96, 87, 77 and 81) and `83` is the average of the third column's grades (70, 90, 90 and 82). Again, NumPy does *not* display trailing 0s to the right of the decimal point in `'83.'`. Also note that it *does* display all element values in the same field width, which is why `'83.'` is followed by two spaces.

Similarly, specifying `axis=1` performs the calculation on all the *column* values within each individual row. To calculate each student's average grade for all exams, we can use:

[lick here to view code image](#)

```
In [11]: grades.mean(axis=1)
```

```
Out[11]: array([84.33333333, 92.33333333, 87.        , 87.66666667])
```

This produces four averages—one each for the values in each row. So `84.33333333` is

the average of row 0's grades (87, 96 and 70), and the other averages are for the remaining rows.

NumPy arrays have many more calculation methods. For the complete list, see

<https://docs.scipy.org/doc/numpy/reference/arrays.ndarray.html>

7.9 UNIVERSAL FUNCTIONS

NumPy offers dozens of standalone **universal functions** (or **ufuncs**) that perform various element-wise operations. Each performs its task using one or two array or array-like (such as lists) arguments. Some of these functions are called when you use operators like + and * on arrays. Each returns a new array containing the results.

Let's create an array and calculate the square root of its values, using the **sqrt universal function**:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: numbers = np.array([1, 4, 9, 16, 25, 36])

In [3]: np.sqrt(numbers)
Out[3]: array([1., 2., 3., 4., 5., 6.] )
```

Let's add two arrays with the same shape, using the **add universal function**:

[lick here to view code image](#)

```
In [4]: numbers2 = np.arange(1, 7) * 10

In [5]: numbers2
Out[5]: array([10, 20, 30, 40, 50, 60])

In [6]: np.add(numbers, numbers2)
Out[6]: array([11, 24, 39, 56, 75, 96])
```

The expression `np.add(numbers, numbers2)` is equivalent to:

```
numbers + numbers2
```

Broadcasting with Universal Functions

Let's use the **multiply universal function** to multiply every element of `numbers2` by the scalar value 5:

[lick here to view code image](#)

```
In [7]: np.multiply(numbers2, 5)
Out[7]: array([ 50, 100, 150, 200, 250, 300])
```

The expression `np.multiply(numbers2, 5)` is equivalent to:

```
numbers2 * 5
```

Let's reshape `numbers2` into a 2-by-3 array, then multiply its values by a one-dimensional array of three elements:

[lick here to view code image](#)

```
In [8]: numbers3 = numbers2.reshape(2, 3)

In [9]: numbers3
Out[9]:
array([[10, 20, 30],
       [40, 50, 60]])

In [10]: numbers4 = np.array([2, 4, 6])

In [11]: np.multiply(numbers3, numbers4)
Out[11]:
array([[ 20,  80, 180],
       [ 80, 200, 360]])
```

This works because `numbers4` has the same length as each row of `numbers3`, so NumPy can apply the multiply operation by treating `numbers4` as if it were the following array:

```
array([[2, 4, 6],
       [2, 4, 6]])
```

If a universal function receives two arrays of different shapes that do not support broadcasting, a `ValueError` occurs. You can view the broadcasting rules at:

Other Universal Functions

The NumPy documentation lists universal functions in five categories—math, trigonometry, bit manipulation, comparison and floating point. The following table lists some functions from each category. You can view the complete list, their descriptions and more information about universal functions at:

<https://docs.scipy.org/doc/numpy/reference/ufuncs.html>

NumPy universal functions

Math—add, subtract, multiply, divide, remainder, exp, log, sqrt, power, and more.

Trigonometry—sin, cos, tan, hypot, arcsin, arccos, arctan, and more.

Bit manipulation—bitwise_and, bitwise_or, bitwise_xor, invert, left_shift and right_shift-.

Comparison—greater, greater_equal, less, less_equal, equal, not_equal, logical_and, logical_or-, logical_xor, logical_not, minimum, maximum, and more.

Floating point—floor, ceil, isinf, isnan, fabs, trunc, and more.

7.10 INDEXING AND SLICING

One-dimensional `arrays` can be indexed and sliced using the same syntax and techniques we demonstrated in the “Sequences: Lists and Tuples” chapter. Here, we focus on `array`-specific indexing and slicing capabilities.

Indexing with Two-Dimensional `arrays`

To select an element in a two-dimensional array, specify a tuple containing the element's row and column indices in square brackets (as in snippet [4]):

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: grades = np.array([[87, 96, 70], [100, 87, 90],
...:                       [94, 77, 90], [100, 81, 82]])
...:

In [3]: grades
Out[3]:
array([[ 87,  96,  70],
       [100,  87,  90],
       [ 94,  77,  90],
       [100,  81,  82]])

In [4]: grades[0, 1] # row 0, column 1
Out[4]: 96
```

Selecting a Subset of a Two-Dimensional array's Rows

To select a single row, specify only one index in square brackets:

[lick here to view code image](#)

```
In [5]: grades[1]
Out[5]: array([100,  87,  90])
```

To select multiple sequential rows, use slice notation:

[lick here to view code image](#)

```
In [6]: grades[0:2]
Out[6]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

To select multiple non-sequential rows, use a list of row indices:

```
In [7]: grades[[1, 3]]
Out[7]:
array([[100,  87,  90],
       [100,  81,  82]])
```

Selecting a Subset of a Two-Dimensional array's Columns

You can select subsets of the columns by providing a tuple specifying the row(s) and column(s) to select. Each can be a specific index, a slice or a list. Let's select only the elements in the first column:

[lick here to view code image](#)

```
In [8]: grades[:, 0]
Out[8]: array([ 87, 100,  94, 100])
```

The 0 after the comma indicates that we're selecting only column 0. The : before the comma indicates which rows within that column to select. In this case, : is a *slice* representing *all* rows. This also could be a specific row number, a slice representing a subset of the rows or a list of specific row indices to select, as in snippets [5] – [7].

You can select consecutive columns using a slice:

```
In [9]: grades[:, 1:3]
Out[9]:
array([[96, 70],
       [87, 90],
       [77, 90],
       [81, 82]])
```

or specific columns using a *list* of column indices:

```
In [10]: grades[:, [0, 2]]
Out[10]:
array([[ 87,  70],
       [100,  90],
       [ 94,  90],
       [100,  82]])
```

7.11 VIEWS: SHALLOW COPIES

The previous chapter introduced *view objects*—that is, objects that “see” the data in other objects, rather than having their own copies of the data. Views are shallow copies. Various array methods and slicing operations produce views of an array's data.

The array method **view** returns a *new* array object with a *view* of the original array

object's data. First, let's create an array and a view of that array:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: numbers = np.arange(1, 6)

In [3]: numbers
Out[3]: array([1, 2, 3, 4, 5])

In [4]: numbers2 = numbers.view()

In [5]: numbers2
Out[5]: array([1, 2, 3, 4, 5])
```

We can use the built-in `id` function to see that `numbers` and `numbers2` are *different* objects:

```
In [6]: id(numbers)
Out[6]: 4462958592

In [7]: id(numbers2)
Out[7]: 4590846240
```

To prove that `numbers2` views the *same* data as `numbers`, let's modify an element in `numbers`, then display both arrays:

[lick here to view code image](#)

```
In [8]: numbers[1] *= 10

In [9]: numbers2
Out[9]: array([ 1, 20,  3,  4,  5])

In [10]: numbers
Out[10]: array([ 1, 20,  3,  4,  5])
```

Similarly, changing a value in the view also changes that value in the original array:

```
In [11]: numbers2[1] /= 10

In [12]: numbers
Out[12]: array([1, 2, 3, 4, 5])
```

```
In [13]: numbers2
Out[13]: array([1, 2, 3, 4, 5])
```

Slice Views

Slices also create views. Let's make `numbers2` a slice that views only the first three elements of `numbers`:

```
In [14]: numbers2 = numbers[0:3]
In [15]: numbers2
Out[15]: array([1, 2, 3])
```

Again, we can confirm that `numbers` and `numbers2` are different objects with `id`:

```
In [16]: id(numbers)
Out[16]: 4462958592

In [17]: id(numbers2)
Out[17]: 4590848000
```

We can confirm that `numbers2` is a view of *only* the first *three* `numbers` elements by attempting to access `numbers2[3]`, which produces an `IndexError`:

[lick here to view code image](#)

```
In [18]: numbers2[3]
-----
IndexError                                Traceback (most recent call last)
ipython-input-18-582053f52daa> in <module>()
----> 1 numbers2[3]

IndexError: index 3 is out of bounds for axis 0 with size 3
```

Now, let's modify an element both arrays share, then display them. Again, we see that `numbers2` is a view of `numbers`:

[lick here to view code image](#)

```
In [19]: numbers[1] *= 20

In [20]: numbers
Out[20]: array([1, 2, 3, 4, 5])
```

```
In [21]: numbers2
Out[21]: array([ 1, 40,  3])
```

7.12 DEEP COPIES

Though views are *separate* array objects, they save memory by sharing element data from other arrays. However, when sharing *mutable* values, sometimes it's necessary to create a **deep copy** with *independent* copies of the original data. This is especially important in multi-core programming, where separate parts of your program could attempt to modify your data at the same time, possibly corrupting it.

The **array method copy** returns a new array object with a *deep copy* of the original array object's data. First, let's create an array and a deep copy of that array:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: numbers = np.arange(1, 6)

In [3]: numbers
Out[3]: array([1, 2, 3, 4, 5])

In [4]: numbers2 = numbers.copy()

In [5]: numbers2
Out[5]: array([1, 2, 3, 4, 5])
```

To prove that `numbers2` has a separate copy of the data in `numbers`, let's modify an element in `numbers`, then display both arrays:

[lick here to view code image](#)

```
In [6]: numbers[1] *= 10

In [7]: numbers
Out[7]: array([ 1, 20,  3,  4,  5])

In [8]: numbers2
Out[8]: array([ 1,  2,  3,  4,  5])
```

As you can see, the change appears only in `numbers`.

Module `copy`—Shallow vs. Deep Copies for Other Types of Python Objects

In previous chapters, we covered *shallow copying*. In this chapter, we've covered how to *deep copy* array objects using their `copy` method. If you need deep copies of other types of Python objects, pass them to the `copy` module's `deepcopy` function.

7.13 RESHAPING AND TRANSPOSING

We've used array method `reshape` to produce two-dimensional arrays from one-dimensional ranges. NumPy provides various other ways to reshape arrays.

`reshape` vs. `resize`

The array methods `reshape` and `resize` both enable you to change an array's dimensions. Method `reshape` returns a *view* (shallow copy) of the original array with the new dimensions. It does *not* modify the original array:

[lick here to view code image](#)

```
In [1]: import numpy as np

In [2]: grades = np.array([[87, 96, 70], [100, 87, 90]])

In [3]: grades
Out[3]:
array([[ 87,  96,  70],
       [100,  87,  90]])

In [4]: grades.reshape(1, 6)
Out[4]: array([[ 87,  96,  70, 100,  87,  90]])

In [5]: grades
Out[5]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

Method `resize` *modifies the original array's shape*:

[lick here to view code image](#)

```
In [6]: grades.resize(1, 6)

In [7]: grades
Out[7]: array([[ 87,  96,  70, 100,  87,  90]])
```

flatten vs. ravel

You can take a multidimensional array and flatten it into a single dimension with the methods **flatten** and **ravel**. Method `flatten` *deep copies* the original array's data:

[lick here to view code image](#)

```
In [8]: grades = np.array([[87, 96, 70], [100, 87, 90]])

In [9]: grades
Out[9]:
array([[ 87,  96,  70],
       [100,  87,  90]])

In [10]: flattened = grades.flatten()

In [11]: flattened
Out[11]: array([ 87,  96,  70, 100,  87,  90])

In [12]: grades
Out[12]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

To confirm that `grades` and `flattened` do *not* share the data, let's modify an element of `flattened`, then display both arrays:

[lick here to view code image](#)

```
In [13]: flattened[0] = 100

In [14]: flattened
Out[14]: array([100,  96,  70, 100,  87,  90])

In [15]: grades
Out[15]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

Method `ravel` produces a *view* of the original array, which *shares* the `grades` array's data:

[lick here to view code image](#)

```
In [16]: raveled = grades.ravel()
```

```
In [17]: raveled
Out[17]: array([ 87,  96,  70, 100,  87,  90])

In [18]: grades
Out[18]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

To confirm that `grades` and `raveled` *share* the same data, let's modify an element of `raveled`, then display both arrays:

[lick here to view code image](#)

```
In [19]: raveled[0] = 100

In [20]: raveled
Out[20]: array([100,  96,  70, 100,  87,  90])

In [21]: grades
Out[21]:
array([[100,  96,  70],
       [100,  87,  90]])
```

Transposing Rows and Columns

You can quickly **transpose** an array's rows and columns—that is “flip” the array, so the rows become the columns and the columns become the rows. The **T attribute** returns a transposed *view* (shallow copy) of the array. The original `grades` array represents two students' grades (the rows) on three exams (the columns). Let's transpose the rows and columns to view the data as the grades on three exams (the rows) for two students (the columns):

```
In [22]: grades.T
Out[22]:
array([[100, 100],
       [ 96,  87],
       [ 70,  90]])
```

Transposing does *not* modify the original array:

```
In [23]: grades
Out[23]:
array([[100,  96,  70],
       [100,  87,  90]])
```

Horizontal and Vertical Stacking

You can combine arrays by adding more columns or more rows—known as *horizontal stacking* and *vertical stacking*. Let's create another 2-by-3 array of grades:

[lick here to view code image](#)

```
In [24]: grades2 = np.array([[94, 77, 90], [100, 81, 82]])
```

Let's assume `grades2` represents three additional exam grades for the two students in the `grades` array. We can combine `grades` and `grades2` with NumPy's **`hstack` (horizontal stack) function** by passing a tuple containing the arrays to combine. The extra parentheses are required because `hstack` expects one argument:

[lick here to view code image](#)

```
In [25]: np.hstack((grades, grades2))
Out[25]:
array([[100, 96, 70, 94, 77, 90],
       [100, 87, 90, 100, 81, 82]])
```

Next, let's assume that `grades2` represents two more students' grades on three exams. In this case, we can combine `grades` and `grades2` with NumPy's **`vstack` (vertical stack) function**:

[lick here to view code image](#)

```
In [26]: np.vstack((grades, grades2))
Out[26]:
array([[100, 96, 70],
       [100, 87, 90],
       [ 94, 77, 90],
       [100, 81, 82]])
```

7.14 INTRO TO DATA SCIENCE: PANDAS SERIES AND DATAFRAMES

NumPy's `array` is optimized for homogeneous numeric data that's accessed via integer indices. Data science presents unique demands for which more customized data structures are required. Big data applications must support mixed data types, customized indexing, missing data, data that's not structured consistently and data that

needs to be manipulated into forms appropriate for the databases and data analysis packages you use.

Pandas is the most popular library for dealing with such data. It provides two key collections that you'll use in several of our Intro to Data Science sections and throughout the data science case studies—**Series** for one-dimensional collections and **DataFrames** for two-dimensional collections. You can use pandas' **MultiIndex** to manipulate multi-dimensional data in the context of `Series` and `DataFrames`.

Wes McKinney created pandas in 2008 while working in industry. The name pandas is derived from the term “panel data,” which is data for measurements over time, such as stock prices or historical temperature readings. McKinney needed a library in which the same data structures could handle both time- and non-time-based data with support for data alignment, missing data, common database-style data manipulations, and more.⁴

⁴ McKinney, Wes. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*, pp. 123165. Sebastopol, CA: O'Reilly Media, 2018.

NumPy and pandas are intimately related. `Series` and `DataFrames` use arrays “under the hood.” `Series` and `DataFrames` are valid arguments to many NumPy operations. Similarly, arrays are valid arguments to many `Series` and `DataFrame` operations.

Pandas is a massive topic—the PDF of its documentation⁵ is over 2000 pages. In this and the next chapters' Intro to Data Science sections, we present an introduction to pandas. We discuss its `Series` and `DataFrames` collections, and use them in support of data preparation. You'll see that `Series` and `DataFrames` make it easy for you to perform common tasks like selecting elements a variety of ways, filter/map/reduce operations (central to functional-style programming and big data), mathematical operations, visualization and more.

⁵ For the latest pandas documentation, see

<http://pandas.pydata.org/pandas-docs/stable/>.

7.14.1 pandas `Series`

A **Series** is an enhanced one-dimensional array. Whereas arrays use only zero-based integer indices, `Series` support custom indexing, including even non-integer indices like strings. `Series` also offer additional capabilities that make them more

convenient for many data-science oriented tasks. For example, `Series` may have missing data, and many `Series` operations ignore missing data by default.

Creating a `Series` with Default Indices

By default, a `Series` has integer indices numbered sequentially from 0. The following creates a `Series` of student grades from a list of integers:

[lick here to view code image](#)

```
In [1]: import pandas as pd

In [2]: grades = pd.Series([87, 100, 94])
```

The initializer also may be a tuple, a dictionary, an array, another `Series` or a single value. We'll show a single value momentarily.

Displaying a `Series`

Pandas displays a `Series` in two-column format with the indices *left aligned* in the left column and the values *right aligned* in the right column. After listing the `Series` elements, pandas shows the data type (`dtype`) of the underlying array's elements:

```
In [3]: grades
Out[3]:
0      87
1     100
2      94
dtype: int64
```

Note how easy it is to display a `Series` in this format, compared to the corresponding code for displaying a list in the same two-column format.

Creating a `Series` with All Elements Having the Same Value

You can create a `Series` of elements that all have the same value:

[lick here to view code image](#)

```
In [4]: pd.Series(98.6, range(3))
Out[4]:
0     98.6
1     98.6
```

```
2      98.6
dtype: float64
```

The second argument is a one-dimensional iterable object (such as a list, an `array` or a `range`) containing the `Series`' indices. The number of indices determines the number of elements.

Accessing a `Series`' Elements

You can access a `Series`'s elements by via square brackets containing an index:

```
In [5]: grades[0]
Out[5]: 87
```

Producing Descriptive Statistics for a `Series`

`Series` provides many methods for common tasks including producing various descriptive statistics. Here we show `count`, `mean`, `min`, `max` and `std` (standard deviation):

[lick here to view code image](#)

```
In [6]: grades.count()
Out[6]: 3

In [7]: grades.mean()
Out[7]: 93.66666666666667

In [8]: grades.min()
Out[8]: 87

In [9]: grades.max()
Out[9]: 100

In [10]: grades.std()
Out[10]: 6.506407098647712
```

Each of these is a functional-style reduction. Calling `Series` method **describe** produces all these stats and more:

[lick here to view code image](#)

```
In [11]: grades.describe()
Out[11]:
```

```
count      3.000000
mean       93.666667
std        6.506407
min        87.000000
25%        90.500000
50%        94.000000
75%        97.000000
max        100.000000
dtype: float64
```

The 25%, 50% and 75% are **quartiles**:

- 50% represents the median of the sorted values.
- 25% represents the median of the first half of the sorted values.
- 75% represents the median of the second half of the sorted values.

For the quartiles, if there are two middle elements, then their average is that quartile's median. We have only three values in our `Series`, so the 25% quartile is the average of 87 and 94, and the 75% quartile is the average of 94 and 100. Together, the **interquartile range** is the 75% quartile minus the 25% quartile, which is another measure of dispersion, like standard deviation and variance. Of course, quartiles and interquartile range are more useful in larger datasets.

Creating a `Series` with Custom Indices

You can specify *custom* indices with the `index` keyword argument:

[lick here to view code image](#)

```
In [12]: grades = pd.Series([87, 100, 94], index=['Wally', 'Eva', 'Sam'])

In [13]: grades
Out[13]:
Wally      87
Eva        100
Sam         94
dtype: int64
```

In this case, we used string indices, but you can use other immutable types, including integers not beginning at 0 and nonconsecutive integers. Again, notice how nicely and concisely pandas formats a `Series` for display.

Dictionary Initializers

If you initialize a `Series` with a dictionary, its keys become the `Series`' indices, and its values become the `Series`' element values:

[lick here to view code image](#)

```
In [14]: grades = pd.Series({'Wally': 87, 'Eva': 100, 'Sam': 94})

In [15]: grades
Out[15]:
Wally      87
Eva       100
Sam        94
dtype: int64
```

Accessing Elements of a `Series` Via Custom Indices

In a `Series` with custom indices, you can access individual elements via square brackets containing a custom index value:

```
In [16]: grades['Eva']
Out[16]: 100
```

If the custom indices are strings that could represent valid Python identifiers, pandas automatically adds them to the `Series` as attributes that you can access via a dot (`.`), as in:

```
In [17]: grades.Wally
Out[17]: 87
```

`Series` also has *built-in* attributes. For example, the **`dtype` attribute** returns the underlying array's element type:

```
In [18]: grades.dtype
Out[18]: dtype('int64')
```

and the **`values` attribute** returns the underlying array:

[lick here to view code image](#)

```
In [19]: grades.values
```

```
Out[19]: array([ 87, 100,  94])
```

Creating a Series of Strings

If a `Series` contains strings, you can use its **`str` attribute** to call string methods on the elements. First, let's create a `Series` of hardware-related strings:

[lick here to view code image](#)

```
In [20]: hardware = pd.Series(['Hammer', 'Saw', 'Wrench'])

In [21]: hardware
Out[21]:
0    Hammer
1         Saw
2     Wrench
dtype: object
```

Note that pandas also *right-aligns* string element values and that the `dtype` for strings is `object`.

Let's call string method `contains` on each element to determine whether the value of each element contains a lowercase 'a':

[lick here to view code image](#)

```
In [22]: hardware.str.contains('a')
Out[22]:
0     True
1     True
2    False
dtype: bool
```

Pandas returns a `Series` containing `bool` values indicating the `contains` method's result for each element—the element at index 2 ('Wrench') does not contain an 'a', so its element in the resulting `Series` is `False`. Note that pandas handles the iteration internally for you—another example of functional-style programming. The `str` attribute provides many string-processing methods that are similar to those in Python's string type. For a list, see: <https://pandas.pydata.org/pandas-ocs/stable/api.html#string-handling>.

The following uses string method `upper` to produce a *new* `Series` containing the

uppercase versions of each element in hardware:

[lick here to view code image](#)

```
In [23]: hardware.str.upper()
Out[23]:
0      HAMMER
1       SAW
2     WRENCH
dtype: object
```

7.14.2 DataFrames

A **DataFrame** is an enhanced two-dimensional array. Like Series, DataFrames can have custom row and column indices, and offer additional operations and capabilities that make them more convenient for many data-science oriented tasks. DataFrames also support missing data. Each column in a DataFrame is a Series. The Series representing each column may contain different element types, as you'll soon see when we discuss loading datasets into DataFrames.

Creating a DataFrame from a Dictionary

Let's create a DataFrame from a dictionary that represents student grades on three exams:

[lick here to view code image](#)

```
In [1]: import pandas as pd

In [2]: grades_dict = {'Wally': [87, 96, 70], 'Eva': [100, 87, 90],
...:                  'Sam': [94, 77, 90], 'Katie': [100, 81, 82],
...:                  'Bob': [83, 65, 85]}
...:

In [3]: grades = pd.DataFrame(grades_dict)

In [4]: grades
Out[4]:
   Wally  Eva  Sam  Katie  Bob
0     87  100   94    100   83
1     96   87   77     81   65
2     70   90   90     82   85
```

Pandas displays DataFrames in tabular format with the indices *left aligned* in the

index column and the remaining columns' values *right aligned*. The dictionary's *keys* become the column names and the *values* associated with each key become the element values in the corresponding column. Shortly, we'll show how to "flip" the rows and columns. By default, the row indices are auto-generated integers starting from 0.

Customizing a DataFrame's Indices with the `index` Attribute

We could have specified custom indices with the `index` keyword argument when we created the `DataFrame`, as in:

[lick here to view code image](#)

```
pd.DataFrame(grades_dict, index=['Test1', 'Test2', 'Test3'])
```

Let's use the **`index` attribute** to change the `DataFrame`'s indices from sequential integers to labels:

[lick here to view code image](#)

```
In [5]: grades.index = ['Test1', 'Test2', 'Test3']

In [6]: grades
Out[6]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

When specifying the indices, you must provide a one-dimensional collection that has the same number of elements as there are *rows* in the `DataFrame`; otherwise, a `ValueError` occurs. `Series` also provides an **`index` attribute** for changing an existing `Series`' indices.

Accessing a DataFrame's Columns

One benefit of pandas is that you can quickly and conveniently look at your data in many different ways, including selecting portions of the data. Let's start by getting Eva's grades by name, which displays her column as a `Series`:

```
In [7]: grades['Eva']
Out[7]:
Test1    100
```

```
Test2      87
Test3      90
Name: Eva, dtype: int64
```

If a DataFrame's column-name strings are valid Python identifiers, you can use them as attributes. Let's get Sam's grades with the `Sam` attribute:

```
In [8]: grades.Sam
Out[8]:
Test1      94
Test2      77
Test3      90
Name: Sam, dtype: int64
```

Selecting Rows via the `loc` and `iloc` Attributes

Though DataFrames support indexing capabilities with `[]`, the pandas documentation recommends using the attributes `loc`, `iloc`, `at` and `iat`, which are optimized to access DataFrames and also provide additional capabilities beyond what you can do only with `[]`. Also, the documentation states that indexing with `[]` *often* produces a copy of the data, which is a logic error if you attempt to assign new values to the DataFrame by assigning to the result of the `[]` operation.

You can access a row by its label via the DataFrame's **`loc` attribute**. The following lists all the grades in the row 'Test1':

[lick here to view code image](#)

```
In [9]: grades.loc['Test1']
Out[9]:
Wally      87
Eva        100
Sam         94
Katie      100
Bob         83
Name: Test1, dtype: int64
```

You also can access rows by integer zero-based indices using the **`iloc` attribute** (the `i` in `iloc` means that it's used with integer indices). The following lists all the grades in the second row:

[lick here to view code image](#)

```
In [10]: grades.iloc[1]
Out[10]:
Wally    96
Eva      87
Sam      77
Katie    81
Bob      65
Name: Test2, dtype: int64
```

Selecting Rows via Slices and Lists with the `loc` and `iloc` Attributes

The index can be a *slice*. When using slices containing labels with `loc`, the range specified *includes* the high index ('Test3'):

[lick here to view code image](#)

```
In [11]: grades.loc['Test1':'Test3']
Out[11]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65
Test3	70	90	90	82	85

When using slices containing integer indices with `iloc`, the range you specify *excludes* the high index (2):

[lick here to view code image](#)

```
In [12]: grades.iloc[0:2]
Out[12]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test2	96	87	77	81	65

To select *specific rows*, use a *list* rather than slice notation with `loc` or `iloc`:

[lick here to view code image](#)

```
In [13]: grades.loc[['Test1', 'Test3']]
Out[13]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test3	70	90	90	82	85


```
In [14]: grades.iloc[[0, 2]]
```

```
Out[14]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87	100	94	100	83
Test3	70	90	90	82	85

Selecting Subsets of the Rows and Columns

So far, we've selected only *entire* rows. You can focus on small subsets of a `DataFrame` by selecting rows *and* columns using two slices, two lists or a combination of slices and lists.

Suppose you want to view only Eva's and Katie's grades on `Test1` and `Test2`. We can do that by using `loc` with a slice for the two consecutive rows and a list for the two non-consecutive columns:

[lick here to view code image](#)

```
In [15]: grades.loc['Test1':'Test2', ['Eva', 'Katie']]
Out[15]:
```

	Eva	Katie
Test1	100	100
Test2	87	81

The slice `'Test1':'Test2'` selects the rows for `Test1` and `Test2`. The list `['Eva', 'Katie']` selects only the corresponding grades from those two columns.

Let's use `iloc` with a list and a slice to select the first and third tests and the first three columns for those tests:

[lick here to view code image](#)

```
In [16]: grades.iloc[[0, 2], 0:3]
Out[16]:
```

	Wally	Eva	Sam
Test1	87	100	94
Test3	70	90	90

Boolean Indexing

One of pandas' more powerful selection capabilities is **Boolean indexing**. For example, let's select all the A grades—that is, those that are greater than or equal to 90:

[lick here to view code image](#)

```
In [17]: grades[grades >= 90]
Out[17]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	NaN	100.0	94.0	100.0	NaN
Test2	96.0	NaN	NaN	NaN	NaN
Test3	NaN	90.0	90.0	NaN	NaN

Pandas checks every grade to determine whether its value is greater than or equal to 90 and, if so, includes it in the new DataFrame. Grades for which the condition is `False` are represented as **NaN (not a number)** in the new DataFrame. NaN is pandas' notation for missing values.

Let's select all the B grades in the range 80–89:

[lick here to view code image](#)

```
In [18]: grades[(grades >= 80) & (grades < 90)]
Out[18]:
```

	Wally	Eva	Sam	Katie	Bob
Test1	87.0	NaN	NaN	NaN	83.0
Test2	NaN	87.0	NaN	81.0	NaN
Test3	NaN	NaN	NaN	82.0	85.0

Pandas Boolean indices combine multiple conditions with the Python operator `&` (bitwise AND), *not* the `and` Boolean operator. For `or` conditions, use `|` (bitwise OR). NumPy also supports Boolean indexing for arrays, but always returns a one-dimensional array containing only the values that satisfy the condition.

Accessing a Specific DataFrame Cell by Row and Column

You can use a DataFrame's `at` and `iat` attributes to get a single value from a DataFrame. Like `loc` and `iloc`, `at` uses labels and `iat` uses integer indices. In each case, the row and column indices must be separated by a comma. Let's select Eva's Test2 grade (87) and Wally's Test3 grade (70)

[lick here to view code image](#)

```
In [19]: grades.at['Test2', 'Eva']
Out[19]: 87

In [20]: grades.iat[2, 0]
Out[20]: 70
```


You also can assign new values to specific elements. Let's change Eva's Test2 grade to 100 using `at`, then change it back to 87 using `iat`:

[lick here to view code image](#)

```
In [21]: grades.at['Test2', 'Eva'] = 100
```

```
In [22]: grades.at['Test2', 'Eva']  
Out[22]: 100
```

```
In [23]: grades.iat[1, 2] = 87
```

```
In [24]: grades.iat[1, 2]  
Out[24]: 87.0
```

Descriptive Statistics

Both `Series` and `DataFrames` have a **describe method** that calculates basic descriptive statistics for the data and returns them as a `DataFrame`. In a `DataFrame`, the statistics are calculated by column (again, soon you'll see how to flip rows and columns):

[lick here to view code image](#)

```
In [25]: grades.describe()
```

```
Out[25]:
```

	Wally	Eva	Sam	Katie	Bob
count	3.000000	3.000000	3.000000	3.000000	3.000000
mean	84.333333	92.333333	87.000000	87.666667	77.666667
std	13.203535	6.806859	8.888194	10.692677	11.015141
min	70.000000	87.000000	77.000000	81.000000	65.000000
25%	78.500000	88.500000	83.500000	81.500000	74.000000
50%	87.000000	90.000000	90.000000	82.000000	83.000000
75%	91.500000	95.000000	92.000000	91.000000	84.000000
max	96.000000	100.000000	94.000000	100.000000	85.000000

As you can see, `describe` gives you a quick way to summarize your data. It nicely demonstrates the power of array-oriented programming with a clean, concise functional-style call. Pandas handles internally all the details of calculating these statistics for each column. You might be interested in seeing similar statistics on test-by-test basis so you can see how all the students performs on Tests 1, 2 and 3—we'll show how to do that shortly.

By default, pandas calculates the descriptive statistics with floating-point values and

displays them with six digits of precision. You can control the precision and other default settings with pandas' **set_option function**:

[lick here to view code image](#)

```
In [26]: pd.set_option('precision', 2)

In [27]: grades.describe()
Out[27]:
```

	Wally	Eva	Sam	Katie	Bob
count	3.00	3.00	3.00	3.00	3.00
mean	84.33	92.33	87.00	87.67	77.67
std	13.20	6.81	8.89	10.69	11.02
min	70.00	87.00	77.00	81.00	65.00
25%	78.50	88.50	83.50	81.50	74.00
50%	87.00	90.00	90.00	82.00	83.00
75%	91.50	95.00	92.00	91.00	84.00
max	96.00	100.00	94.00	100.00	85.00

For student grades, the most important of these statistics is probably the mean. You can calculate that for each student simply by calling `mean` on the `DataFrame`:

```
In [28]: grades.mean()
Out[28]:
```

Wally	84.33
Eva	92.33
Sam	87.00
Katie	87.67
Bob	77.67

dtype: float64

In a moment, we'll show how to get the average of all the students' grades on each test in one line of additional code.

Transposing the `DataFrame` with the `T` Attribute

You can quickly **transpose** the rows and columns—so the rows become the columns, and the columns become the rows—by using the **T attribute**:

[lick here to view code image](#)

```
In [29]: grades.T
Out[29]:
```

	Test1	Test2	Test3
Wally	87	96	70
Eva	100	87	90